9.4 Regex: The Complete Rules

Regex (regular expressions) describes a *pattern* of text so you can find, match, or replace it. Syntax varies slightly between languages (JavaScript, Python, PCRE, and others). The rules below are the shared core.

Patterns below are written the way you type them in an R string for `stringr`. A digit is `\\d`. A word character is `\\w`. In `str_replace()`, a captured group is `\\1`.

1. Literals

Most characters match themselves.

- `cat` matches "cat" inside "concatenate"
- Matching is case-sensitive by default

2. Metacharacters (special characters)

These have special meaning:

```
. ^ $ * + ? ( ) [ ] { } | \
```

To match one literally, type it the way R requires: `"\\."` is a period, `"\\$"` is a dollar sign, and `"\\\\"` is one backslash.

3. The dot

`.` matches **any single character** except a newline (by default).

- `c.t` matches "cat", "cut", "c9t", "c t"

In `stringr`, `regex(p, dotall = TRUE)` lets `.` match a newline too.

4. Character classes

Square brackets match **one character** from a set.

Pattern	Meaning
`[abc]`	a, b, or c
`[a-z]`	any lowercase letter
`[A-Za-z0-9]`	any letter or digit
`[^abc]`	anything **except** a, b, c (the `^` negates when first)
`[.]`	a literal dot (most metacharacters lose their power inside `[]`)

Inside a class, only `] \ ^ -` are special. Put `-` at the start or end to match a literal hyphen: `[a-z-]`.

5. Shorthand classes

Shorthand	Meaning	ASCII picture
`\\d`	digit	`[0-9]`
`\\D`	non-digit	`[^0-9]`
`\\w`	word character	`[A-Za-z0-9_]`
`\\W`	non-word character	`[^A-Za-z0-9_]`
`\\s`	whitespace (space, tab, newline)	`[ \\t\\n\\r\\f\\v]`
`\\S`	non-whitespace	

In `stringr`, `\\d` and `\\w` are Unicode-aware. `\\d` matches decimal digits beyond `0-9`, and `\\w` matches letters such as `é`.

6. Anchors and boundaries

These match a **position**, not a character.

Pattern	Matches
`^`	start of string (or line, with multiline flag)
`$`	end of string (or line)
`\\b`	word boundary (between `\\w` and `\\W`)
`\\B`	not a word boundary

- `^\\d+$` matches a string that is *entirely* digits
- `\\bcat\\b` matches "cat" but not "concatenate"

7. Quantifiers

Quantifiers say **how many times** the preceding item repeats.

Pattern	Meaning
`*`	0 or more
`+`	1 or more
`?`	0 or 1 (optional)
`{3}`	exactly 3
`{3,}`	3 or more
`{3,5}`	between 3 and 5

Examples: `colou?r` matches "color" and "colour"; `\\d{3}-\\d{4}` matches "555-1234".

Greedy vs. lazy vs. possessive

- **Greedy** (default): matches as much as possible. `<.+>` on `<b>hi</b>` matches the **whole** string.
- **Lazy**: add `?` to match as little as possible. `<.+?>` matches just `<b>`.
- **Possessive**: add `+` (e.g. `a++`). It matches greedily and never backtracks. ICU, which `stringr` uses, supports this. JavaScript does not, and Python did not before 3.11.

7b. Quantifiers apply to one thing

A quantifier only affects the item right before it: `ab+` means "a" then one or more "b"s. To repeat multiple characters, use a group: `(ab)+`.

8. Alternation (OR)

`|` means "or."

- `cat|dog` matches "cat" or "dog"
- `gr(a|e)y` matches "gray" or "grey"

Alternation has the **lowest precedence**, so `^cat|dog$` means "starts with cat" OR "ends with dog." Use groups to control scope: `^(cat|dog)$`.

9. Groups

Syntax	Purpose
`(...)`	**Capturing group**: groups items and remembers the match
`(?:...)`	**Non-capturing group**: groups only, does not remember
`(?<name>...)`	**Named capturing group**. `stringr` accepts this form. Python spells it `(?P<name>...)`

Capturing groups are numbered left to right by their opening parenthesis: `(\\d{4})-(\\d{2})-(\\d{2})` on "2026-10-27" gives group 1 = 2026, group 2 = 10, group 3 = 27.

10. Backreferences

Refer back to what a group already matched.

- `(\\w)\\1` matches a doubled letter like "ee" or "ll"
- `(['"]).*?\\1` matches a quoted string with matching quote types
- In R, type `\\k<name>`. Python uses `(?P=name)`.

In `str_replace()`, type `"\\1"`. JavaScript uses `$1` instead.

11. Lookarounds (zero-width assertions)

These check what is nearby **without consuming it**.

Syntax	Name	Meaning
`X(?=Y)`	positive lookahead	X followed by Y
`X(?!Y)`	negative lookahead	X **not** followed by Y
`(?<=Y)X`	positive lookbehind	X preceded by Y
`(?<!Y)X`	negative lookbehind	X **not** preceded by Y

Examples:

- `\\d+(?= dollars)` matches the number in "50 dollars" but not the word
- `(?<=\\$)\\d+` matches digits after a `$` without including it
- `^(?=.*\\d)(?=.*[A-Z]).{8,}$` is a password check: 8+ characters, at least one digit and one capital

In `stringr`, a lookbehind must have a bounded maximum length. `(?<=\\d{2})` and `(?<=a{1,3})` are fine. `(?<=a+)` is not, because `+` has no upper bound.

12. Flags / modifiers

Flag	Effect
`i`	case-insensitive
`g`	global: find all matches, not just the first (JS and others)
`m`	multiline: `^` and `$` match at each line
`s`	dotall: `.` also matches newlines
`x`	extended: allows whitespace and comments in the pattern (Python, PCRE)
`u`	Unicode mode (JS)

`stringr` sets these with `regex()`. `regex(p, ignore_case = TRUE)` is `i`, `multiline = TRUE` is `m`, and `dotall = TRUE` is `s`. There is no `g` flag: `str_extract_all()` and `str_replace_all()` find every match. You can also put `(?i)` at the front of the pattern.

13. Unicode

- `\\p{L}` any letter, `\\p{Lu}` uppercase letter, `\\p{N}` any number
- `\\P{L}` is the negation
- In some languages `\\w` and `\\d` are ASCII-only unless you turn Unicode on. `stringr` already treats them as Unicode

JavaScript needs the `u` flag before Unicode properties work. Python's standard `re` does not support them; the third-party `regex` library does.

14. Escape sequences

`\\n` newline, `\\t` tab, `\\r` carriage return, `\\xHH` hex character, `\\uHHHH` Unicode character, `\\0` null.

15. How the engine thinks

1. **It scans left to right** and returns the first position where the whole pattern matches.
2. **Backtracking**: if a greedy part overshoots and the rest fails, the engine gives characters back and retries.
3. **Catastrophic backtracking**: nested quantifiers like `(a+)+$` can take exponential time on bad input. Avoid them, especially on user input.

16. Practical examples

Goal	Pattern
Integer	`^-?\\d+$`
Decimal	`^-?\\d+(\\.\\d+)?$`
US ZIP code	`^\\d{5}(-\\d{4})?$`
Hex color	`^#(?:[0-9a-fA-F]{3}){1,2}$`
ISO date	`^\\d{4}-\\d{2}-\\d{2}$`
Simple email	`^[\\w.+-]+@[\\w-]+\\.[\\w.-]+$`
Extra whitespace	`\\s{2,}`
HTML tag (simple)	`<([a-z]+)>.*?</\\1>`
Repeated word	`\\b(\\w+)\\s+\\1\\b`

Email and HTML are hard to match completely with regex. These patterns are good enough for cleaning and extracting, not for validating every legal address or parsing a page.

17. Tips for building patterns

1. **Start small**, then build up piece by piece.
2. **Test interactively** with regex101.com. It explains each part. For this course, choose a flavor close to ICU, or test with `str_view()`.
3. **Be specific**: prefer `[^"]*` over `.*` when matching inside quotes.
4. **Anchor** when validating whole strings (`^...$`).
5. **Escape** user-provided text before inserting it into a pattern. In R, `str_escape()` does this.
6. **Know when not to use regex**: nested structures (HTML, JSON, code) are better handled by a real parser.

Quick cheat sheet

```
.        any character (not a newline)     ^     start
\\d       digit                             $     end
\\w       word character                    \\b    word boundary
\\s       whitespace                        |     or
[ ]      one of a set                      ( )   capturing group
[^ ]     not one of a set                  (?: ) non-capturing group
*        0 or more                         \\1    backreference
+        1 or more                         ?     lazy, after a quantifier
?        0 or 1                            {n,m} between n and m
(?=) (?!) (?<=) (?<!)                      lookarounds
```